计算机系统 I：信息表示 (Information Representation) 复习讲义

目录

1 信息表示基础 (Foundations of Information Representation) 2
1.1 信息与信号 (Information & Signals) 2
1.2 外部信息与内部数据 (External Information & Internal Data) 2

2 数制与整数表示 (Radix Systems & Integer Representation) 2
2.1 数制与转换 (Radix Number Systems & Conversions) 2
2.2 整数的内部表示 (Internal Representation of Integers) 3

3 浮点数表示 (Floating-Point Representation) 5
3.1 二进制小数基础 (Fractional Binary Numbers) 5
3.2 IEEE 754 标准 (IEEE 754 Standard) 5
3.3 浮点数的分类 (Categories of Floating-Point Values) 6
3.4 浮点数编码与运算特性 (Encoding and Arithmetic Properties) 7

4 非数值数据与存储 (Non-numeric Data & Storage) 9
4.1 非数值数据的表示 (Representation of Non-numeric Data) 9
4.2 数据宽度与单位 (Data Width and Unit) 9
4.3 主存容量与频率单位 (Units of Capacity and Frequency) 10
4.4 数据存储的字节序 (Byte Ordering) 10

5 全章总结 (Summary) 10

1 信息表示基础 (Foundations of Information Representation)

1.1 信息与信号 (Information & Signals)

- **数字系统 (Digital Systems)**：变量取离散值。二进制值（0 和 1，False/True，Low/High）是数字系统中最常见的表示形式。
- **采用二进制的原因**：物理器件容易实现（例如高低电平、磁场方向、表面凹凸、电荷有无），抗干扰能力强，易于进行逻辑运算。

信号的分类 (Classification of Signals)

按照取值与时间是否连续，信号可分为：

- **模拟信号 (Analog)**：取值和时间均连续。
- **异步数字信号 (Asynchronous)**：取值离散，但时间连续。
- **同步数字信号 (Synchronous)**：取值和时间均离散（数字系统的目标形态）。

在物理实现上，电压信号存在一个**阈值区 (Threshold Region)**，高于/低于阈值分别判定为 1/0，阈值区的存在正是二进制抗干扰能力强的根本原因。

1.2 外部信息与内部数据 (External Information & Internal Data)

从不同视角看，计算机处理的数据可分层理解：

- **外部信息 (External Information)**：文本、图像、声音、视频等，通过输入/输出设备与用户交互，在高级程序员视角下还包括图、树、链表、队列等**结构化数据**，最终都以二进制编码存储。
- **内部数据 (Internal Data)**：从底层程序员 / 硬件设计者视角看，基本类型 (Primitive Types) 分为两大类：
  - **数值型 (Numeric)**：整数（定点数，又分无符号 Unsigned 与有符号 Signed）和实数（浮点数）。
  - **非数值型 (Non-numeric)**：位串 (Bits)、字符 / 字符串 (Character/String)。

2 数制与整数表示 (Radix Systems & Integer Representation)

2.1 数制与转换 (Radix Number Systems & Conversions)

计算机中常见进制包括：二进制 (Base 2)、八进制 (Base 8)、十进制 (Base 10) 和十六进制 (Base 16)。

任意进制到十进制 (R-base to Decimal)

按权展开求和：

$$(Number)_r = \sum_{i=0}^{n-1} A_i \cdot r^i + \sum_{j=-m}^{-1} A_j \cdot r^j$$

例如：$(100101.01)_2 = 1 \times 2^5 + 1 \times 2^2 + 1 \times 2^0 + 1 \times 2^{-2} = 32 + 4 + 1 + 0.25 = (37.25)_{10}$

十进制到任意进制 (Decimal to R-base)

- **整数部分**：除基取余法（Repeated Division），逆序排列。
- **小数部分**：乘基取整法（Repeated Multiplication），顺序排列。

注：小数部分的转换可能会产生无限循环（如将十进制的 *0.65* 转换为二进制），实际应用中需根据位宽进行截断 *(Truncate)* 或舍入 *(Round)*。

二进制与八进制/十六进制的互相转换

- **二进制** ↔ **八进制**：以小数点为界，向两侧每 3 位分为一组，不足补零。
- **二进制** ↔ **十六进制**：以小数点为界，向两侧每 4 位分为一组，不足补零。

2.2 整数的内部表示 (Internal Representation of Integers)

机器数与真值 (Machine Number and True Value)

机器数是数据在计算机内部的二进制编码，真值是其对应的实际数值。对于带符号整数，最高位为符号位（0 正 1 负）。

整数的编码方式 (Encoding Methods for Integers)

四种主要的编码方式

1. **原码 (Sign-and-Magnitude)**：最高位为符号位，其余位表示数值的绝对值。存在 $+0$ 和 -0 两种表示。
2. **反码 (One's Complement)**：正数的反码是其本身；负数的反码是符号位保持不变，其余各位按位取反。
3. **补码 (Two's Complement)**：广泛使用的表示法。正数补码是其本身；负数补码是其**反码加 1**。
   - 只有唯一的 0 表示。
   - n 位补码表示范围：$-2^{n-1} \leq X \leq 2^{n-1}-1$。负数比正数多一个（即 -2^{n-1} 没有对应的正数）。
   - 取反加一等式：$\sim x + 1 = -x$。其原理是 $\sim x + x = 111\ldots1 = -1$。
4. **移码 (Frameshift Code / Biased Notation)**：常用于浮点数的阶码表示。给真值加上一个偏置常数（Bias）得到，等同于补码符号位取反。

补码的数学定义与求补码的快捷方法 (Definition & Shortcut)

从“基的补”角度，补码有两种形式化定义（设 N 为 n 位二进制数）：

- **反基补码 (Diminished Radix Complement)**：对基数 r 即 $(r-1)$ 的补，二进制下为 1's complement，定义为 $(r^n-1)-N$。由于 r^n-1 全为 1，且 $1-0=1$，$1-1=0$，因此等价于**逐位取反**。
- **基补码 (Radix Complement)**：对基数 r 即 r 的补，二进制下为 2's complement，定义为 r^n-N，即反码加 1。

求补码的快捷方法（从最低位向最高位扫描）：保留末尾所有的 0，保留遇到的第一个 1，此后的各位**全部取反**。例如 10010100 的补码为 01101100（下划线 <u>100</u> 保留，左侧取反）。

符号扩展与截断 (Sign Extension and Truncation)

- **符号扩展 (Sign Extension)**：当小字长转换为大字长时。无符号数高位补 0；有符号数高位补符号位（0 或 1）。
- **截断 (Truncating)**：大字长转换为小字长时，直接丢弃高位，保留低位。可能会改变原有数值，相当于对 2^k 取模。

3 浮点数表示 (Floating-Point Representation)

3.1 二进制小数基础 (Fractional Binary Numbers)

分数二进制的按权展开：

$$\sum_{k=-j}^{i} b_k \times 2^k$$

小数点右侧的位代表 $2^{-1}(1/2), 2^{-2}(1/4), 2^{-3}(1/8)$ 等。其局限性在于只能精确表示形如 $x/2^k$ 的有理数，其他有理数（如 1/3, 1/10）会产生无限循环位表示。这引出了后续将学习的 IEEE 754 浮点数标准。

3.2 IEEE 754 标准 (IEEE 754 Standard)

为了统一计算机内部小数的表示，IEEE 制定了 754 浮点数标准。浮点数在底层被表示为科学计数法的形式：

$$V = (-1)^s \times M \times 2^E$$

- **符号 (Sign, s)**：决定数值的正负。$s = 0$ 为正，$s = 1$ 为负。
- **尾数 (Significand/Mantissa, M)**：一个二进制小数，决定了数值的精度。
- **阶码 (Exponent, E)**：对基数 2 进行加权，决定了数值的范围。

在计算机存储中，浮点数被划分为三个相连的位字段：

1. **符号位 (`s`)**：直接编码符号 s。
2. **阶码字段 (`exp`)**：编码阶码 E。
3. **小数字段 (`frac`)**：编码尾数 M。

主要精度类型与相对精度 (Precision Types & Relative Precision)

- **单精度 (Single Precision, `float`)**：共 32 位。分配为：1 位符号位，8 位阶码，23 位小数。
  - **偏置常数 (Bias)**：$2^{8-1} - 1 = 127$。
  - **相对精度**：最小分辨率约 2^{-23}，等效于 $23 \times \log_{10} 2 \approx 23 \times 0.3 \approx 7$ 位十进制有效数字。
- **双精度 (Double Precision, `double`)**：共 64 位。分配为：1 位符号位，11 位阶码，52 位小数。
  - **偏置常数 (Bias)**：$2^{11-1} - 1 = 1023$。
  - **相对精度**：最小分辨率约 2^{-52}，等效于 $52 \times \log_{10} 2 \approx 52 \times 0.3 \approx 16$ 位十进制有效数字。

关于偏置常数的设计目的 (Why Bias?)

引入偏置常数的核心目的是将带正负号的实际指数 e 映射为**无符号整数**阶码 E ($E = e + \text{Bias}$)。这种设计避免了使用补码表示指数所带来的大小比较复杂性。计算公式采取 $2^{k-1}-1$（取了剔除全 0 和全 1 特殊值后的中间区间），使得指数分布更平衡，并具有符合科学计算需求的轻微正数倾向性。

3.3 浮点数的分类 (Categories of Floating-Point Values)

根据阶码字段 (`exp`) 的位模式，浮点数被解析为以下三种不同的情况：

规格化值 (Normalized Values)

这是最普遍的情况。当 `exp` 的位模式**既不全为 0，也不全为 1** 时，该数值为规格化数。

- **真实阶码** E：$E = \text{exp} - \text{Bias}$。
- **真实尾数** M：$M = 1 + f$。这里隐含了一个开头的 “1”（因为非零二进制数的最高有效位总是 1，所以不需要存），尾数值相当于 $1.f_{n-1}\ldots f_0$。

非规格化值 (Denormalized Values)

当 `exp` 的位模式**全为 0** 时，该数值为非规格化数。主要用于表示 0 以及非常接近 0 的数（渐进式下溢）。

- **真实阶码** E：$E = 1 - \text{Bias}$（注意这里不是 $0 - \text{Bias}$，这种设计是为了与规格化数平滑过渡）。
- **真实尾数** M：$M = f$。不再隐含开头的 1，即 $0.f_{n-1}\ldots f_0$。
- **零的表示**：当 `exp` 全为 0 且 `frac` 全为 0 时，表示零。根据符号位的不同，有 $+0.0$ 和 -0.0 之分。

特殊值 (Special Values)

当 `exp` 的位模式**全为 1** 时，表示特殊值。

- **无穷大 (Infinity)**：当 `frac` 全为 0 时，表示无穷大。$s = 0$ 为 $+\infty$，$s = 1$ 为 $-\infty$。通常产生于除以零或溢出等操作。
- **NaN (Not a Number)**：当 `frac` 不全为 0 时，表示 “不是一个数”。通常产生于无意义的数学运算，如 $\sqrt{-1}$ 或 $\infty - \infty$。

微型浮点数示例 (Tiny Floating Point Example)

为便于手算理解三类取值的分界，常用一个 8 位微型浮点格式：1 位符号、4 位阶码（Bias $= 2^{4-1} - 1 = 7$）、3 位小数。它与 IEEE 标准的结构完全相同，同样包含规格化、非规格化、0、∞ 与 NaN。

- 最小非规格化正数：`0 0000 001`，$E = 1 - 7 = -6$，$M = 1/8$，值为 $\frac{1}{8} \times 2^{-6} = 2^{-9}$。
- 最大非规格化数与最小规格化数在数轴上**平滑衔接**（这正是非规格化数取 $E = 1 - \text{Bias}$ 的目的）。
- 最大规格化数：`0 1110 111`，$E = 14 - 7 = 7$，$M = 1.875$，值为 $1.875 \times 2^7 = 240$。

3.4 浮点数编码与运算特性 (Encoding and Arithmetic Properties)

IEEE 754 在设计位排布时采用了 [符号位 | 阶码 | 尾数] 的固定顺序，这赋予了浮点数在硬件底层处理上的极大优势：

- **零的统一 (FP Zero Same as Integer Zero)**：浮点数 +0.0 的所有二进制位全为 0，这与无符号整数 `0` 的二进制表示完全一致。因此在进行内存级操作（如 C 语言中的 `memset` 清零）时，可以直接按整数逻辑处理。
- **无符号整数比较优化 (Unsigned Integer Comparison)**：由于阶码位于高位，且已被 Bias 机制转换为无符号数，绝大部分浮点数的大小逻辑（如非规格化数与规格化数、规格化数与无穷大之间）与其二进制按无符号整数解析出的大小逻辑是一致的。这意味着 CPU 可以直接复用**整数比较器电路**来比较同号浮点数，极大优化了硬件性能（需特殊处理的仅有符号位、$-0.0 = 0.0$ 判定以及 NaN）。

舍入 (Rounding)

由于二进制浮点数的表示精度有限，当某个实数无法精确表示时，必须采取舍入操作。IEEE 754 定义了四种舍入模式，其中默认且最常用的是**向偶数舍入 (Round-to-Even)**。

它试图找到最接近的匹配值。这种方法可以避免传统的 “逢五进一” 在统计中产生系统性的偏大偏差。具体规则如下（假设要舍入到保留位数的最后一位）：

- **小于一半 ($< 1/2$)**：直接向下舍入（舍弃后续位）。
- **大于一半 ($> 1/2$)**：向上舍入（保留位的最低有效位加 1）。
- **恰好一半 ($= 1/2$)**：即待舍弃部分恰好是 $100\ldots_2$ 的形式。此时观察保留部分的**最低有效位**：
  - 若为 **1 (奇数)**，则向上进位，使其变为 0。
  - 若为 **0 (偶数)**，则直接舍弃，保持不变。
  - 核心原则：最终结果的最低有效位必然为 0（即 “偶数”）。

示例：舍入到最近的 1/4（保留二进制小数点后 2 位）

为了清晰说明，我们观察以下数值如何舍入到二进制小数点后两位（即根据小数点后第三位及其之后的位来决定动作）。

原始值	二进制表示	舍弃部分分析	舍入动作	舍入结果
$2\frac{3}{32}$	$10.00\mathbf{011}_2$	011_2 (小于 100_2)	向下舍入 (Down)	10.00_2 ($=2$)
$2\frac{3}{16}$	$10.00\mathbf{110}_2$	110_2 (大于 100_2)	向上进位 (Up)	10.01_2 ($=2\frac{1}{4}$)
$2\frac{7}{8}$	$10.11\mathbf{100}_2$	100_2 (等于 100_2)	**向偶数舍入** (末位是 1, 进位变 0)	11.00_2 ($=3$)
$2\frac{5}{8}$	$10.10\mathbf{100}_2$	100_2 (等于 100_2)	**向偶数舍入** (末位是 0, 直接舍弃)	10.10_2 ($=2\frac{1}{2}$)

浮点数运算的特性：缺乏结合律 (Lack of Associativity)

由于浮点数存在精度限制（Precision Limit），其加法运算在计算机底层不满足数学上的结合律：

$$(x+y)+z \neq x+(y+z)$$

示例分析

设 $x=-1.5\times10^{38}$, $y=1.5\times10^{38}$, $z=1.0$。

- **计算** $(x+y)+z$：先计算 $x+y=0.0$，再加上 z，结果为 **1.0**。
- **计算** $x+(y+z)$：由于单精度浮点数仅有约 7 位有效数字，在计算 $y+z$ 时，1.0 相对于 1.5×10^{38} 过于微小，在对阶操作中会被舍弃（Rounding off），导致 $y+z\approx1.5\times10^{38}$。最后加上 x，结果为 **0.0**。

结论与影响：在高性能计算和并行计算（Parallel Computing）中，由于计算任务常被拆分并行执行，操作顺序的不确定性可能导致微小的计算误差。在编写涉及大规模数据求和的算法（如深度学习中的梯度累加）时，需特别注意此类精度风险。

数值误差引发的真实灾难 (Disasters Attributable to Bad Numerics)

浮点 / 定点误差并非纯理论问题，历史上曾酿成严重事故：

- **爱国者导弹拦截失败 (1991)**：海湾战争期间，爱国者导弹系统因把开机以来的时间用浮点累加（0.1 秒无法精确表示），长时间运行后**累积时间误差**，导致未能拦截飞毛腿导弹，造成 28 名士兵阵亡。
- **阿丽亚娜 5 号火箭爆炸 (1996)**：惯性参考系统将一个 64 位浮点数转换为 16 位有符号整数时**溢出**，引发连锁故障，火箭升空 40 秒后解体，损失约 5 亿美元。

4 非数值数据与存储 (Non-numeric Data & Storage)

4.1 非数值数据的表示 (Representation of Non-numeric Data)

计算机不仅需要处理数值，还需要处理逻辑值、字符等非数值型数据。这些数据在底层同样必须被编码为二进制形式。

十进制数的二进制编码 (Binary Coded Decimal, BCD)

为了方便计算机处理十进制数据，通常使用 BCD 码来表示十进制数字。

- **有权 BCD 码 (Weighted BCD Code)**：每一位二进制数都有一个固定的权重。最常用的是 **8421 码**。
- **无权 BCD 码 (No weight BCD Code)**：没有固定权重的编码方案，常用的有 **余 3 码 (Excess-3 code)** 和 **格雷码 (Gray code)**。

逻辑值的表示 (Representation of Logical Values)

在通常情况下，一个字（Word）或其他可寻址的数据单元被视为一个整体。但在某些逻辑操作中，我们需要将一个 n 位的数据视为由 n 个独立的 1 位数据组成，每一位都是 0 或 1，从而表示 n 个逻辑值。

字符与字符串 (Alphanumeric Codes)

- **ASCII 码**：美国信息交换标准代码。它使用 7 位二进制来表示字符，包含 94 个图形打印字符（如大写字母、小写字母、数字 0-9 等）和 34 个非打印控制字符（如退格、回车等）。在 ASCII 中，大小写字母之间的转换只需要翻转第 6 位比特即可。
- **Unicode**：将 ASCII 扩展到了 16 位，提供了 65,536 个通用字符代码空间，主要用于统一编码世界上不同语言的文字。
- **汉字编码**：汉字系统包含输入码（用于键盘输入）、机内码（用于系统内部存储与处理，通常分为区码和位码）以及字形码（用于显示和打印）。

4.2 数据宽度与单位 (Data Width and Unit)

- **比特 (Bit)**：构成二进制数的最小单位。
- **字节 (Byte)**：1 字节等于 8 比特。
- **字长 (Word Size)**：机器处理整数型数据（包括内存地址）的标称大小。
  - 32 位机器的字长为 4 字节，这限制了其最大寻址空间为 4GB。
  - 64 位机器的高端系统使用 8 字节字长，如 x86-64 架构支持 48 位地址线，寻址空间可达 256 TB。

在 C 语言中，不同的数据类型在不同架构下的宽度有所区别。例如，在典型的 32 位和 64 位机器上：

- `char`：1 字节。
- `int` 和 `float`：通常为 4 字节。
- `double`：通常为 8 字节。
- 指针 `(pointer)`：在 32 位机器上为 4 字节，在 64 位机器上为 8 字节。

特别注意二进制单位（基于 $2^{10} = 1024$）与十进制单位（基于 $10^3 = 1000$）的区别：

- KB (10^3) vs KiB (2^{10}), MB (10^6) vs MiB (2^{20}), GB (10^9) vs GiB (2^{30})
- 更高单位：T (Tera, 2^{40}), P (Peta, 2^{50}), E (Exa, 2^{60}), Z (Zetta, 2^{70}), Y (Yotta, 2^{80})

4.3 主存容量与频率单位 (Units of Capacity and Frequency)

- **容量单位**（基于 2 的幂）：1 KB = 2^{10} bytes，1 MB = 2^{20} bytes，1 GB = 2^{30} bytes 等。
- **频率与传输速率单位**（基于 10 的幂）：1 kb/s = 10^3 bps，1 Mb/s = 10^6 bps 等。

4.4 数据存储的字节序 (Byte Ordering)

对于跨越多个字节的多字节数据（如一个 32 位的整型），在内存中如何排列这些字节是一个关键的体系结构设计选择。

假设我们有一个 4 字节的十六进制变量 `x = 0x01234567`，它的起始存储地址为 `0x100`：

1. **大端序 (Big Endian)**：最高有效字节（MSB, `01`）存放在最低地址 (`0x100`)。后续字节依次存放在递增的地址中（`01 23 45 67`）。常用于 Sun SPARC、PPC Mac 以及互联网网络传输协议中。

2. **小端序 (Little Endian)**：最低有效字节（LSB, `67`）存放在最低地址 (`0x100`)。后续字节依次存放在递增的地址中（`67 45 23 01`）。目前主流的 x86 架构均采用小端序。

字节序类型	0x100	0x101	0x102	0x103
Big Endian (大端序)	01	23	45	67
Little Endian (小端序)	67	45	23	01

5 全章总结 (Summary)

本章围绕“信息如何在计算机中表示”展开：无论是数值、字符、逻辑值，还是内存中的多字节数据，本质上都需要被编码为二进制位串。复习时可以抓住以下几条主线：

1. **二进制是共同基础**：数字系统使用离散信号表示信息，二进制因易实现、抗干扰、便于逻辑运算而成为计算机内部表示的核心。

2. **数制转换依赖位权**：任意进制到十进制用按权展开，十进制到其他进制分别处理整数部分和小数部分；二进制与八进制、十六进制之间可按 3 位或 4 位分组快速转换。

3. **整数编码关注符号和范围**：原码、反码、补码和移码解决有符号数表示问题，其中补码最常用，必须掌握其表示范围、符号扩展和截断后的数值变化。

4. **浮点数是近似表示**：IEEE 754 通过符号位、阶码和尾数字段表达 $(-1)^s \times M \times 2^E$，并用规格化值、非规格化值和特殊值覆盖不同数值场景。

5. **浮点运算不等同于实数运算**：

   - 浮点运算**不满足结合律与分配律**。
   - 在进行相等判定（如 `if(a == b)`）时极其危险，通常应使用误差范围（Epsilon）进行判定。

6. **非数值数据也需要编码**：BCD、逻辑值、ASCII、Unicode 与汉字编码体现了不同类型信息到二进制形式的映射。

7. **存储细节影响程序理解**：字节、字长、容量单位、频率单位和大小端字节序都会影响数据在内存中的组织方式，也会影响底层调试和跨平台程序行为。

> “计算机中的数据不是原始信息本身，而是经过规则编码后的二进制表示。”